Practical 6: Implementation of SJF Scheduling (Preemptive & Non-Preemptive)

Author: Gaurav Rao

Roll No: 24293916091

Class: CSE A (IInd year)

Date: Feburary 25, 2026

Topic: Implementation of scheduling algo: Round Robin

1. Theory: Shortest Job First (SJF)

Shortest Job First (SJF) is a CPU scheduling algorithm that selects the process with the smallest burst time. It can be implemented in two ways:

- **Non-Preemptive SJF**: Once a process starts, it runs to completion.
- **Preemptive SJF (Shortest Remaining Time First, SRTF)**: If a new process arrives with a shorter burst time than the remaining time of the current process, the CPU is preempted and assigned to the new process.

Pros & Cons

- **Pros**: Minimizes average waiting time.
- **Cons**: May cause starvation for long processes.
- **Used In**: Batch systems, where jobs are known in advance.

.

2. Example Scenario (Solved)

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Non-Preemptive SJF Solution

- At time 0: Only P1 has arrived, so P1 runs (0-7).
- At time 7: P2, P3, P4 have arrived. Shortest burst is P3 (1), so P3 runs (7-8).
- At time 8: P2 and P4 remain. P2 (4), P4 (4). P2 runs (8-12).
- At time 12: P4 runs (12-16).

Process	Waiting	Turnaround
P1	0	7
P2	6	10
P3	3	4
P4	7	11

Preemptive SJF (SRTF) Solution

- At time 0: P1 starts (remaining 7).
- At time 2: P2 arrives (remaining 4). P2 < P1, so P2 runs (2-4).

- At time 4: P3 arrives (remaining 1). P3 < P2, so P3 runs (4-5).
- At time 5: P4 arrives (remaining 4). P2 resumes (5-7).
- At time 7: P2 finishes. P1 resumes (7-14).
- At time 14: P4 runs (14-18).

Process	Waiting	Turnaround
P1	7	14
P2	3	7
P3	0	1
P4	9	13

.

3. C Implementation

Non-Preemptive SJF

```c
#include <stdio.h>
#define MAX 10

struct Process {
  int pid, arrival, burst, waiting, turnaround, completed;
};

void inputProcesses(struct Process p[], int n) {
  for (int i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("P%d Arrival: ", i + 1);
    scanf("%d", &p[i].arrival);
    printf("P%d Burst: ", i + 1);
    scanf("%d", &p[i].burst);
    p[i].completed = 0;
  }
}

void calculateSJFNonPreemptive(struct Process p[], int n) {
  int completed = 0, time = 0;
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int idx = -1, min = 1e9;
    for (int i = 0; i < n; i++) {
      if (!p[i].completed && p[i].arrival <= time && p[i].burst < min) {
        min = p[i].burst; idx = i;
      }
    }
    if (idx == -1) { time++; continue; }
    printf("P%d\t\t%d\t%d\n", p[idx].pid, time, time + p[idx].burst);
    p[idx].waiting = time - p[idx].arrival;
    if (p[idx].waiting < 0) p[idx].waiting = 0;
    time += p[idx].burst;
    p[idx].turnaround = p[idx].waiting + p[idx].burst;
    p[idx].completed = 1;
```

```
    completed++;
  }
}

void printTable(struct Process p[], int n) {
  float avg_wt = 0, avg_tat = 0;
  printf("\nPID\tAT\tBT\tWT\tTAT\n");
  for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival, p[i].burst,
p[i].waiting, p[i].turnaround);
    avg_wt += p[i].waiting;
    avg_tat += p[i].turnaround;
  }
  printf("\nAverage Waiting Time: %.2f", avg_wt / n);
  printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);
}

int main() {
  struct Process p[MAX];
  int n;
  printf("Non-Preemptive SJF\nEnter number of processes: ");
  scanf("%d", &n);
  inputProcesses(p, n);
  calculateSJFNonPreemptive(p, n);
  printTable(p, n);
  return 0;
}
```

Preemptive SJF (SRTF)

```
#include <stdio.h>
#define MAX 10

struct Process {
  int pid, arrival, burst, remaining, waiting, turnaround, completed;
};

void inputProcesses(struct Process p[], int n) {
  for (int i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("P%d Arrival: ", i + 1);
    scanf("%d", &p[i].arrival);
    printf("P%d Burst: ", i + 1);
    scanf("%d", &p[i].burst);
    p[i].remaining = p[i].burst;
    p[i].completed = 0;
  }
}

void calculateSRTF(struct Process p[], int n) {
  int completed = 0, time = 0, last_idx = -1, start_time = 0;
```

```
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int idx = -1, min = 1e9;
    for (int i = 0; i < n; i++) {
      if (!p[i].completed && p[i].arrival <= time && p[i].remaining <
min) {
        min = p[i].remaining; idx = i;
      }
    }
    if (idx != last_idx) {
      if (last_idx != -1) printf("P%d\t\t%d\t%d\n", p[last_idx].pid,
start_time, time);
      start_time = time;
      last_idx = idx;
    }
    if (idx == -1) { time++; continue; }
    p[idx].remaining--;
    time++;
    if (p[idx].remaining == 0) {
      p[idx].completed = 1;
      completed++;
      p[idx].turnaround = time - p[idx].arrival;
      p[idx].waiting = p[idx].turnaround - p[idx].burst;
    }
  }
  if (last_idx != -1) printf("P%d\t\t%d\t%d\n", p[last_idx].pid,
start_time, time);
}

void printTable(struct Process p[], int n) {
  float avg_wt = 0, avg_tat = 0;
  printf("\nPID\tAT\tBT\tWT\tTAT\n");
  for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival, p[i].burst,
p[i].waiting, p[i].turnaround);
    avg_wt += p[i].waiting;
    avg_tat += p[i].turnaround;
  }
  printf("\nAverage Waiting Time: %.2f", avg_wt / n);
  printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);
}

int main() {
  struct Process p[MAX];
  int n;
  printf("Preemptive SJF (SRTF)\nEnter number of processes: ");
  scanf("%d", &n);
  inputProcesses(p, n);
  calculateSRTF(p, n);
  printTable(p, n);
  return 0;
```

```
}
```

.

4. Output Screenshots

- **Non-Preemptive SJF Output:**

```
~/Documents/CW/OS/practicalsAndAssignments/practicals/pr6
❯ ./a.out
Non-Preemptive SJF
Enter number of processes: 5
P1 Arrival: 2
P1 Burst: 4
P2 Arrival: 2
P2 Burst: 3
P3 Arrival: 1
P3 Burst: 6
P4 Arrival: 3
P4 Burst: 1
P5 Arrival: 2
P5 Burst: 5

Order of Execution:
process_name    start   end
P3              1       7
P4              7       8
P2              8       11
P1              11      15
P5              15      20

PID     AT      BT      WT      TAT
P1      2       4       9       13
P2      2       3       6       9
P3      1       6       0       6
P4      3       1       4       5
P5      2       5       13      18

Average Waiting Time: 6.40
Average Turnaround Time: 10.20
```

- **Preemptive SJF Output:**

```
~/Documents/CW/OS/practicalsAndAssignments/practicals/pr6
❯ ./a.out
Preemptive SJF (SRTF)
Enter number of processes: 5
P1 Arrival: 2
P1 Burst: 4
P2 Arrival: 2
P2 Burst: 3
P3 Arrival: 1
P3 Burst: 6
P4 Arrival: 3
P4 Burst: 1
P5 Arrival: 2
P5 Burst: 5

Order of Execution:
process_name    start   end
P3              1       2
P2              2       3
P4              3       4
P2              4       6
P1              6       10
P3              10      15
P5              15      20

PID     AT      BT      WT      TAT
P1      2       4       4       8
P2      2       3       1       4
P3      1       6       8       14
P4      3       1       0       1
P5      2       5       13      18

Average Waiting Time: 5.20
Average Turnaround Time: 9.00
```